Build a Microservices CRUD App

React · ASP.NET Core · SQL Server · Docker · CI/CD · Render

A hands-on lab · use → / ← or Space to navigate

GOAL

What we're building

- Users **register / log in** → get a **JWT** (Authentication service)
- Logged-in users **Create / Read / Update / Delete** products (CRUD service)
- The Product API is **protected** — no valid token, no access

Frontend

React (Vite) single-page app

2 backends

ASP.NET Core Web APIs

Database

SQL Server (one DB per service)

Ops

Docker · GitHub Actions · Render

CONCEPT

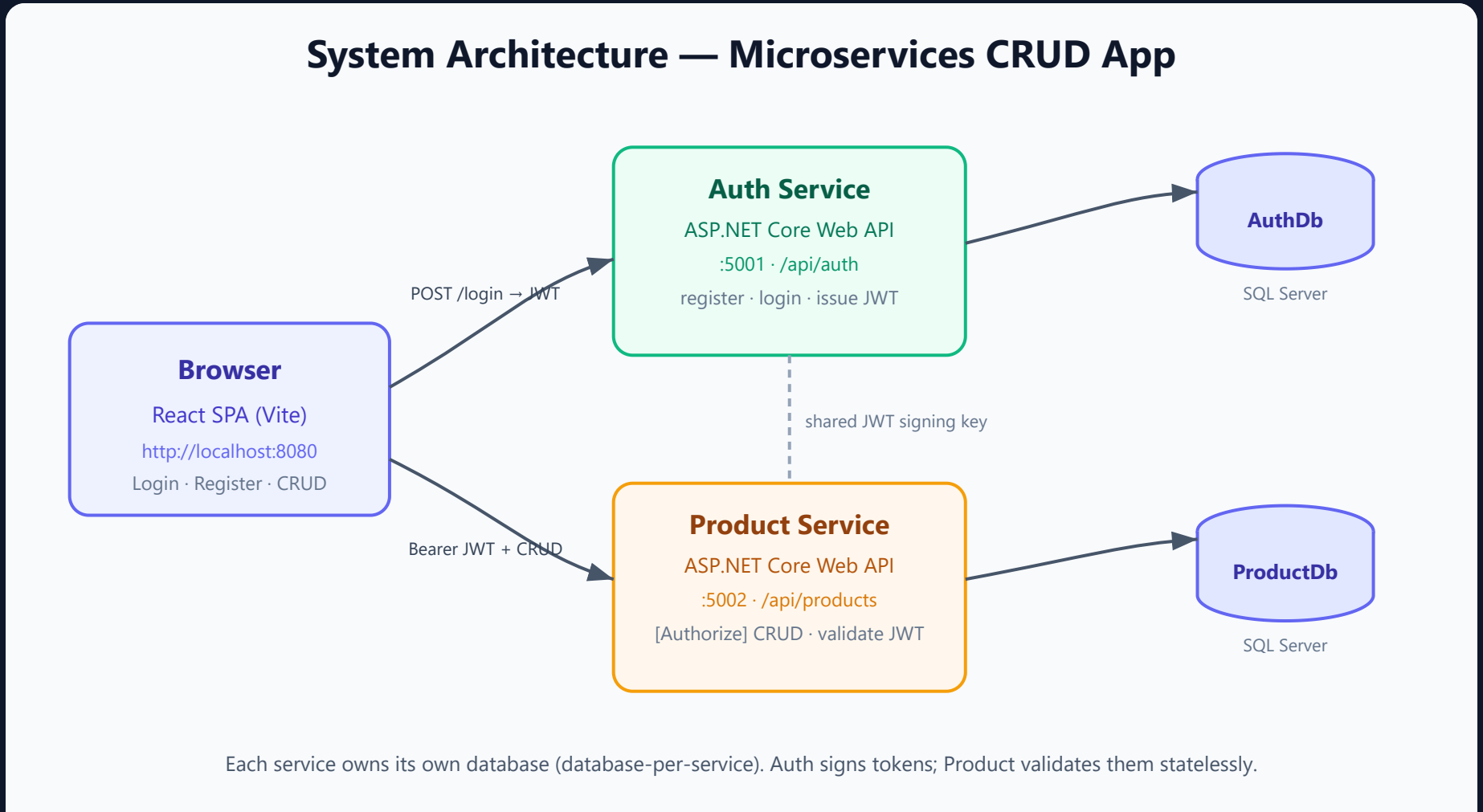
What is a microservice?

- A **small, independently deployable** service that owns **one** job
- **Database-per-service:** each service owns its own data — no shared tables
- Services talk over **HTTP/REST**

Here: an Auth service and a Product service, each with its own SQL Server database.

ARCHITECTURE

How it fits together



Each service owns its own database (database-per-service). Auth signs tokens; Product validates them statelessly.

THE KEY IDEA

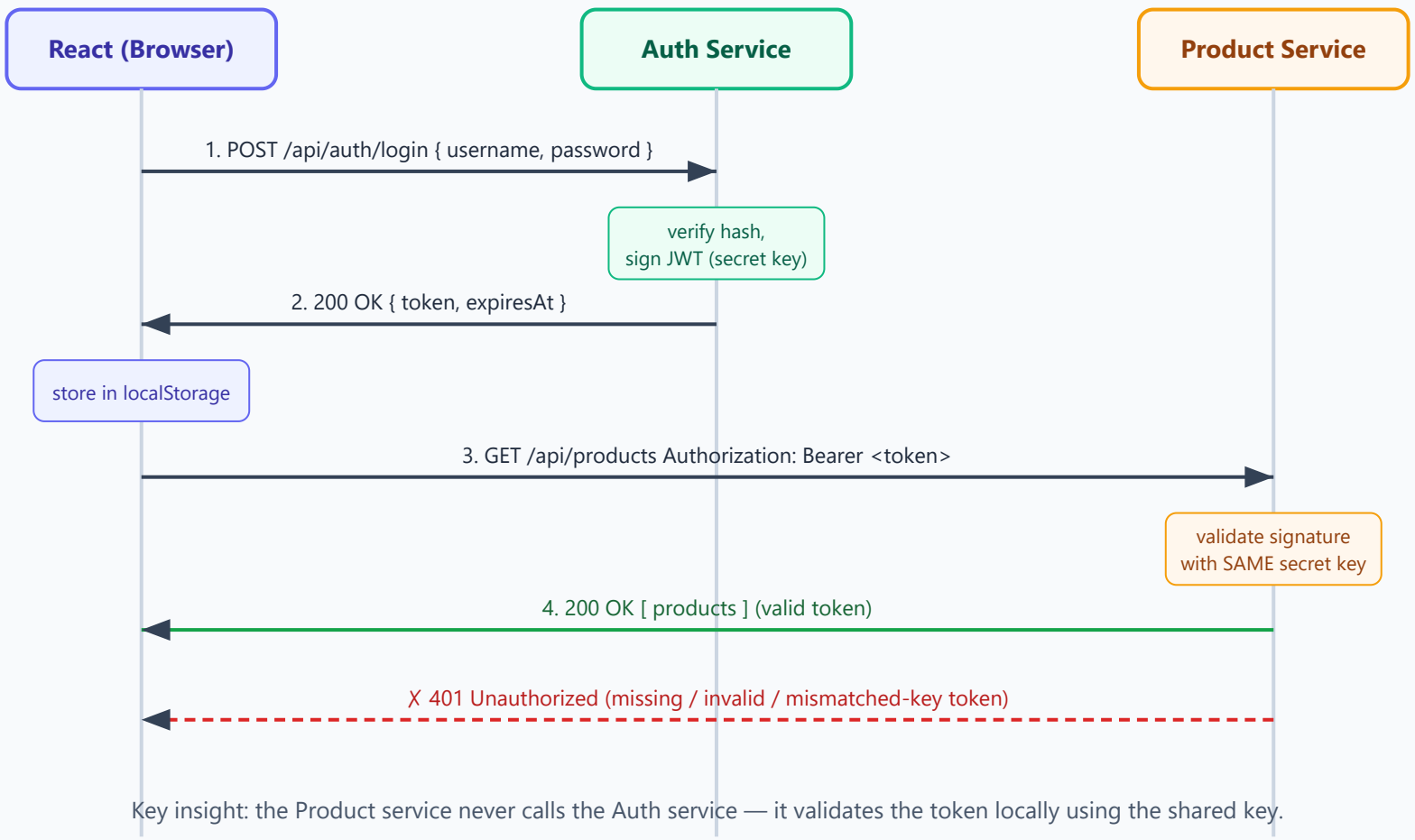
JWT across services

1. Auth service **signs** a token with a secret key on login
2. React stores it and sends `Authorization: Bearer <token>`
3. Product service **validates** it with the **same key** — no call back to Auth

> 🔑 **Golden rule:** `Jwt__Key`, `Jwt__Issuer`, `Jwt__Audience` must be **identical** in both services — or you get **401**.

FLOW

Login → token → protected call



PART A

SQL Server in Docker

No manual install — run it as a container:

```
docker run -e "ACCEPT_EULA=Y" \
  -e "MSSQL_SA_PASSWORD=Your_password123" \
  -p 1433:1433 --name lab-sqlserver -d \
  mcr.microsoft.com/mssql/server:2022-latest
```

Each service creates its own DB on first run via `EnsureCreated()`.

PART B

Auth service (ASP.NET)

- `POST /api/auth/register` and `POST /api/auth/login`
- Passwords hashed with **PBKDF2** + random salt — never stored raw
- `TokenService` signs the JWT; returns `{ token, expiresAt }`
- EF Core + SQL Server → `AuthDb`

PART C

Product CRUD service

- Full CRUD on `/api/products`
- `[Authorize]` on the controller → every route needs a valid JWT
- `AddJwtBearer(...)` validates issuer, audience, lifetime, signature
- Own database → `ProductDb` (seeded with 2 sample products)

Demo: `GET /api/products` → **401** → Authorize → **200** 🎉

PART D

React frontend

api.js

fetch wrapper; attaches the Bearer token

AuthContext

stores user + token in localStorage

Pages

Login · Register · Products (CRUD)

ProtectedRoute

redirects anonymous users to /login

⚠️ Vite reads `VITE_*` env vars at **build time**.

The app

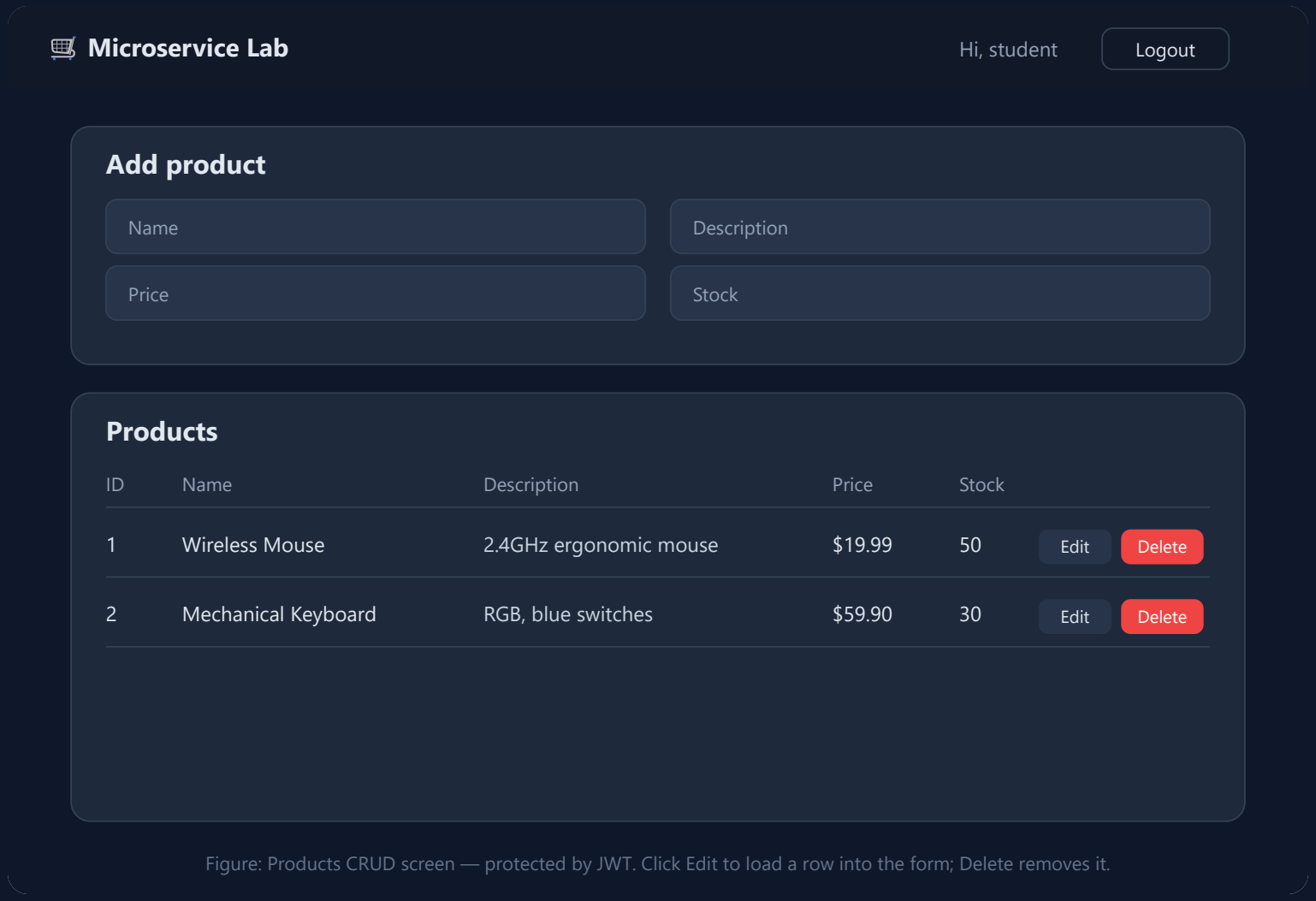


Figure: Products CRUD screen — protected by JWT. Click Edit to load a row into the form; Delete removes it.

PART F

Docker Compose — one command

```
docker compose up --build
```

- Frontend → `localhost:8080`
- Auth → `localhost:5001` · Product → `localhost:5002`

Inside the compose network the DB host is `sqlserver`, **not** `localhost`. #1 gotcha!

DOCKER

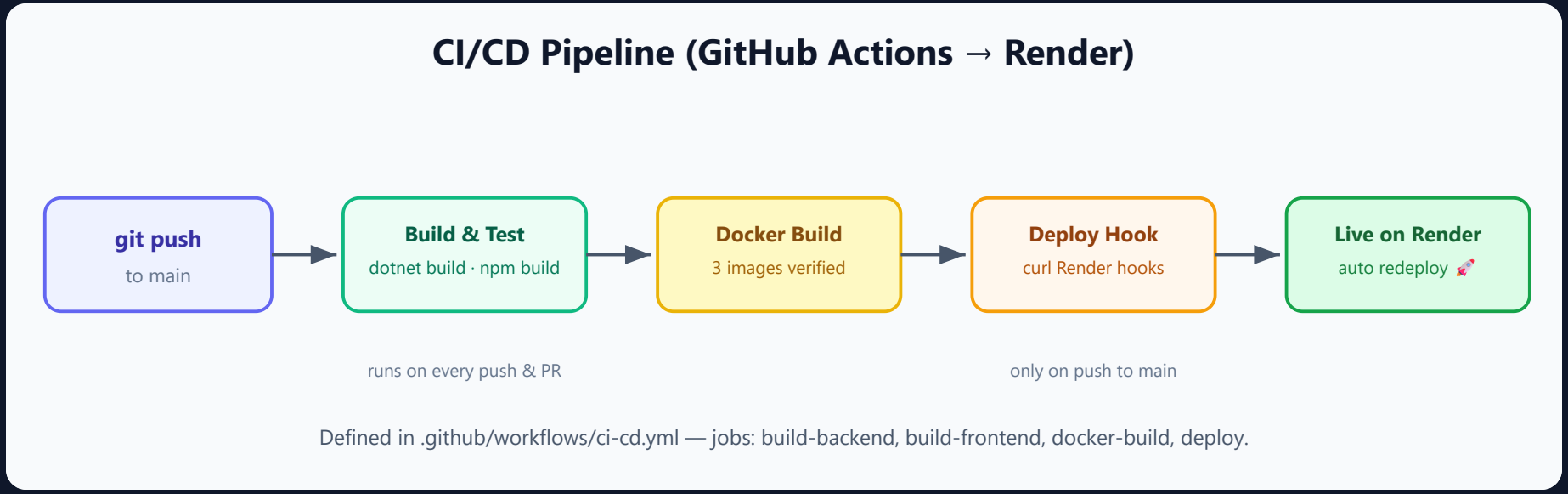
Multi-stage builds

- **Build stage:** heavy SDK / Node — compiles the app
- **Runtime stage:** tiny image — runs only the output
- Result: smaller, faster, more secure images

Frontend: Node builds → **nginx** serves the static files.

PART G

CI/CD with GitHub Actions



Build & test the services, build Docker images, then deploy on push to `main`.

PART H

Deploy to Render

- Two ASP.NET services as **Docker web services**
- React as a **static site** (build → publish `dist`)
- Config via **environment variables**: connection string, JWT key, CORS
- SQL Server hosted externally (Azure SQL free) — Render gets the connection string

Deploy hooks + GitHub secrets → every push auto-deploys. 🚀

DON'T GET STUCK

Top 3 pitfalls

1. **401 everywhere:** JWT key/issuer/audience differ between services
2. **CORS error:** frontend URL not in `AllowedOrigins`
3. **DB unreachable in Docker:** used `localhost` instead of `sqlserver`

You built a real microservices app

2 ASP.NET services · JWT auth · SQL Server · React · Docker · CI/CD · Render

Questions? Open `README.md` for the full step-by-step.